

Phase 3 Recap & Mini System

Phase 3 · Advanced · Estimated time: 2 hours

1. Concept From First Principles: Putting It All Together

If any chapter in this Phase felt shaky on this recap, treat that as useful signal, not failure — go back to that specific chapter before Phase 4, because Phase 4 (Interview-Ready) assumes every idea from Phases 1 through 3 is already load-bearing, the way this Phase assumed Phase 2's mental models were solid.

This chapter has no new concepts. Its job is to force you to combine most of Phase 3 into one coherent, senior-level design. We'll design a **multi-region, fault-tolerant payment ledger system** — the kind of system underlying Razorpay, PhonePe, or a Saudi fintech like Tamara, and a genuinely representative "advanced" HLD interview problem.

Step 1 — Clarify requirements

- Functional: record debit/credit transactions against user accounts; support balance queries; support transaction history and audit trail.
- Functional: settlement/reconciliation reporting across the full transaction history (an OLAP-shaped need).
- Non-functional: the ledger must never lose an acknowledged transaction, and must never double-process one (strong correctness).
- Non-functional: survive a full regional outage with an RTO of minutes and an RPO of seconds.
- Non-functional: operate across India, UAE, and Saudi Arabia, respecting data residency for Saudi customer financial data.

Step 2 — High-level architecture

Given the correctness and audit-trail requirements, the ledger's write path uses **Event Sourcing** (Chapter 8): every transaction is an immutable event ("AccountDebited," "AccountCredited") appended to an event store, giving the audit trail for free and enabling exact historical reconstruction for disputes and compliance. A **CQRS** read model, kept in sync asynchronously, serves fast balance and transaction-history queries without replaying full event histories on every read.

Step 3 — Consensus, replication, and regional architecture

Given the correctness stakes, the ledger's core write path uses a **Raft-based** (Chapter 2) replicated data store per region/shard, giving automatic, correct leader failover within a region without manual intervention. Given the strong-consistency requirement (a CP choice, Phase 2 Chapter 4), the system runs **active-passive** (Chapter 11) across regions rather than active-active — India runs as the active region for Indian accounts, with a passive standby; similarly for UAE, while Saudi customer financial data is kept within Saudi-region infrastructure specifically to satisfy data-residency requirements, meaning this isn't a single global active-passive pair but a regionally-partitioned active-passive design, avoiding cross-region write conflicts entirely by never needing cross-region writes for a single account

in the first place.

Step 4 — Multi-service consistency and resilience

A transaction touching multiple internal services (ledger write, fraud check, notification) uses an orchestrated **Saga** (Chapter 4) rather than 2PC, given this program's established preference for availability over 2PC's blocking behavior at this throughput. Every downstream call (fraud check, notification service) is wrapped in a **circuit breaker** with dedicated **bulkhead** resource pools (Chapter 14), so a slow fraud-check dependency can't cascade into blocking the core ledger write path. Every operation carries an **idempotency key** (Phase 2, Ch 9), since Sagas and at-least-once event delivery both require it for safety.

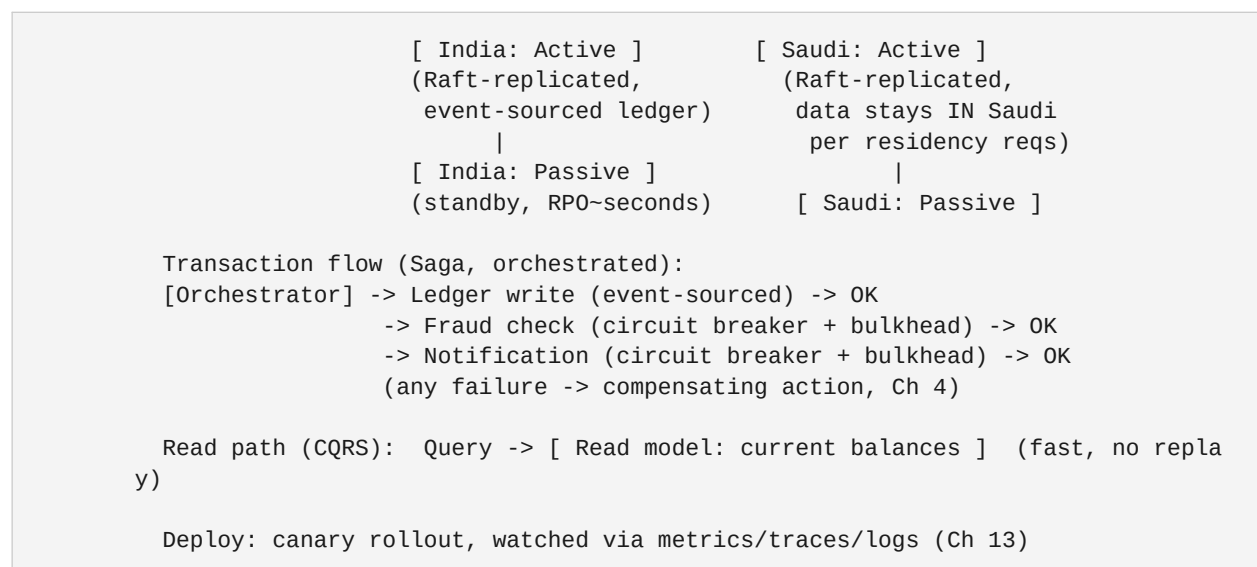
Step 5 — Backup, deployment, and observability

Given the near-zero RPO requirement, the event store's own multi-region replication (not periodic backups alone) is the primary recovery mechanism, with periodic restore drills (Chapter 15) validating the disaster-recovery process actually works. Code deploys to the ledger service use **canary deployment** (Chapter 15), given the severe blast-radius cost of a bad deploy to a financial system, with **distributed tracing and metrics** (Chapter 13) providing the observability signal that makes the canary stage meaningful rather than a false sense of safety.

2. Real-World Example / Case Study

This exact combination — event-sourced ledger, Raft-replicated regional stores, Saga-based multi-service coordination, circuit breakers around external dependencies, canary deployments, and region-aware data residency — describes the general shape of how real payment platforms at Razorpay, PhonePe, and Gulf-region fintech scale are architected, even though the exact technology choices vary company to company.

3. Diagram



4. Trade-offs Table

Decision made	Alternative considered	Why this choice, at this scale
Event Sourcing for the ledger	Store current balance only	Audit trail and point-in-time reconstruction are hard requirements
Raft-replicated regional stores	Manual failover, no consensus	Automatic, correct leader failover without human intervention
Active-passive per region (not global active-active)	Global active-active	Avoids cross-region write conflicts for CP-critical ledger data
Saga (orchestrated) for multi-service writes	Two-Phase Commit	Avoids 2PC's blocking behavior at real throughput
Canary deployment for ledger service	Deploy to 100% at once	Severe blast-radius cost of a bad deploy justifies gradual rollout

5. Common Interview Questions

Q1: "Walk me through your architecture for this system, end to end."

Model approach: Follow the same order used in this chapter — requirements, then the architecture/consensus/replication decisions, then multi-service resilience, then operational concerns — narrating the reasoning behind each decision as you go.

Q2: "Why event sourcing here specifically, and not for every service in the system?"

Model approach: Tie the choice directly to the ledger's genuine audit-trail and point-in-time reconstruction requirements (Chapter 8's honest cost/benefit framing), and state plainly that a simpler internal service without those requirements wouldn't justify the same complexity.

Q3: "What's the very first thing that breaks if you tried to make this globally active-active instead?"

Model approach: Reason from Chapter 11's write-conflict problem — two regions accepting writes to the same account's balance near-simultaneously would require conflict resolution logic that's very hard to make safe for financial correctness, which is exactly why the design chose active-passive per region instead.

6. Practice Exercises (Capstone)

- 1 Without looking back, design this payment ledger system from scratch: requirements, architecture diagram, and identify which four Phase 3 chapters you leaned on most.
- 2 Add a new requirement: the system must support real-time fraud alerts within 2 seconds of a suspicious transaction. Explain what changes, referencing Chapter 7's batch vs. stream processing.
- 3 A regulator requires proof that your disaster recovery plan actually works. Describe the restore drill (Chapter 15) you'd run and what evidence it would produce.

7. Curated YouTube Videos

Language	Video & Channel	Why it helps
English	"Complete System Design Roadmap for 90 Days FAANG Interviews"	Good sanity check of how Phase 3's advanced topics fit the broader interview roadmap.
English	"System Design Verticals — How Big Tech Engineers Think About Architecture"	Reinforces end-to-end architectural reasoning across everything this Phase covered.
Hindi	"Complete System Design Roadmap 2026 HLD & LLD" — CodeHelp	Full Hindi roadmap, worth revisiting now that Phase 3 is complete.
Hindi	"System Design Complete Course 2026" (Hindi + English, 10+ hrs)	Comprehensive bilingual review covering every Phase 3 topic in one course.

8. Key Takeaways

- A repeatable order works for any advanced HLD problem: requirements -> architecture -> consistency/consensus -> resilience -> operations.
- Event Sourcing and CQRS earn their complexity specifically where audit trails and multiple read patterns are genuine requirements.
- Consensus algorithms (Raft) automate correct failover; Sagas avoid 2PC's blocking cost for multi-service writes.
- Regional architecture decisions (active-active vs. active-passive) must account for both consistency needs and data-residency law.
- Circuit breakers and bulkheads contain blast radius at the service level; canary deployments contain it at the deploy level.
- Observability (Chapter 13) is a prerequisite for resilience patterns and canary deployments to actually work, not an afterthought.
- You are now ready for Phase 4, which turns everything from Phases 1-3 into a repeatable interview framework and a set of fully-worked mock design problems.